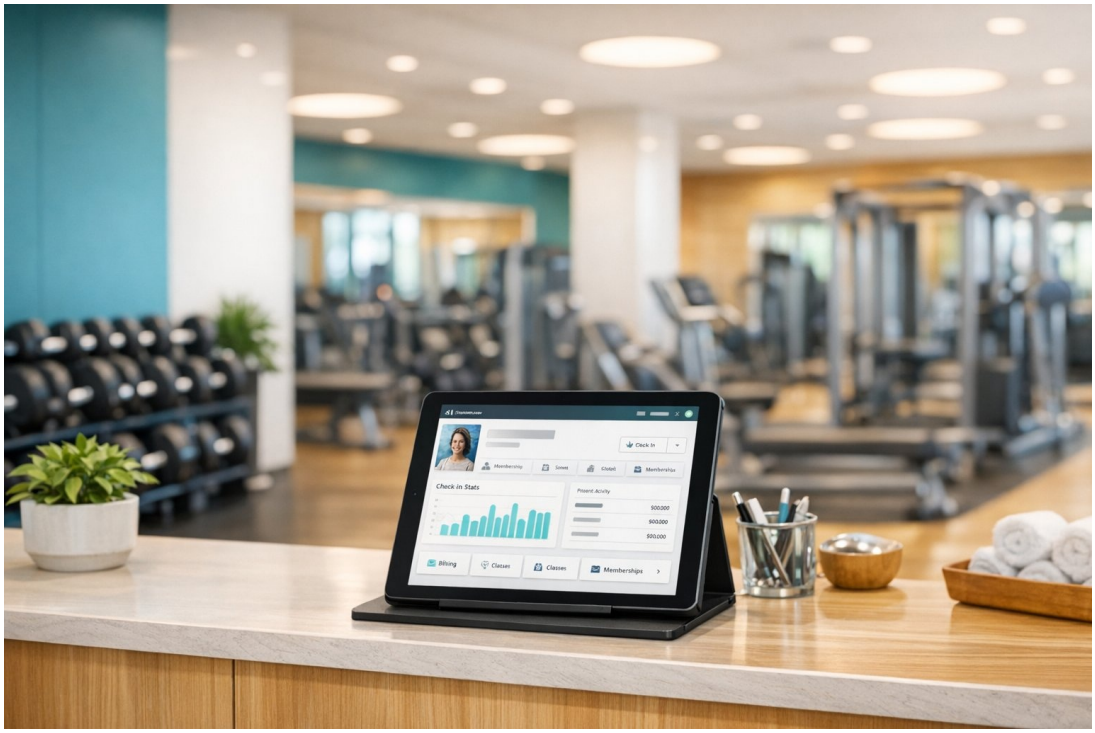
SII - Letnia Akademia Talentów 2026

SII Polska

Gym Membership Management System - Technical Task

Context

Create an application for managing gym memberships, including gyms, membership plans and members.



Functional requirements

1. Each gym must have a unique name.
2. A gym has an address and a phone number.
3. Membership plans belong to a gym. A gym can have multiple membership plans.

4. Each membership plan:
    1. has a name and a type: `BASIC`, `PREMIUM` or `GROUP`
    2. has a monthly price (amount with currency) - the application should not be limited to only one currency
    3. has a duration expressed in months
    4. has a maximum number of members (`maxMembers`)
5. A member subscribes to exactly one membership plan.
6. Each member:
    1. has a full name and an email address
    2. has a membership start date (set automatically upon registration)
    3. has a status: `ACTIVE` or `CANCELLED`
7. A newly registered member starts with `ACTIVE` status.
8. A membership plan cannot accept new members if the number of `ACTIVE` members has reached `maxMembers`.
9. Cancelling a membership changes the member's status to `CANCELLED`. Cancelled members no longer count toward the plan's capacity.
10. [Optional] Revenue report: display total monthly revenue per gym, grouped by currency. Monthly revenue = sum of monthly prices of all `ACTIVE` members for each gym.

Revenue report example:

Gym name	Amount	Currency
FitLife Center	2048.00	EUR
FitLife Center	512.64	GBP
Iron Gym	1024.00	PLN

REST API endpoints

1. Create a new gym.
2. List all gyms.
3. Create a new membership plan for a given gym.
4. List all membership plans for a given gym.
5. Register a new member to a given membership plan (validate capacity).
6. List all members - include the plan name, gym name and status.
7. Cancel a membership.

8. [Optional] Return the revenue report described above.

Non-functional requirements

1. Create only the backend part (no UI is required).
2. The application should expose a REST API.
3. Use Java programming language and Spring framework.
4. Use Maven or Gradle.
5. Use relational in-memory database (e.g. H2).
6. No security features (authorization, authentication, encryption etc.) are required.

Hints

1. Remember that correct operation of the application has a higher priority than completing all the functionality of the system. It is better to write less but well than more and with errors.
2. Think about unusual use cases and test your application.
3. Include a short instruction how to build and run the application with URL's to REST services along with sample queries.
4. Keep the database schema simple.
5. Remember about decimal points in amounts (e.g. `0.99 EUR`).
6. Try to commit regularly, so that you can trace the development of the application.

Submission form

Please submit the result of your work as a GIT repository.

Good luck!

SII